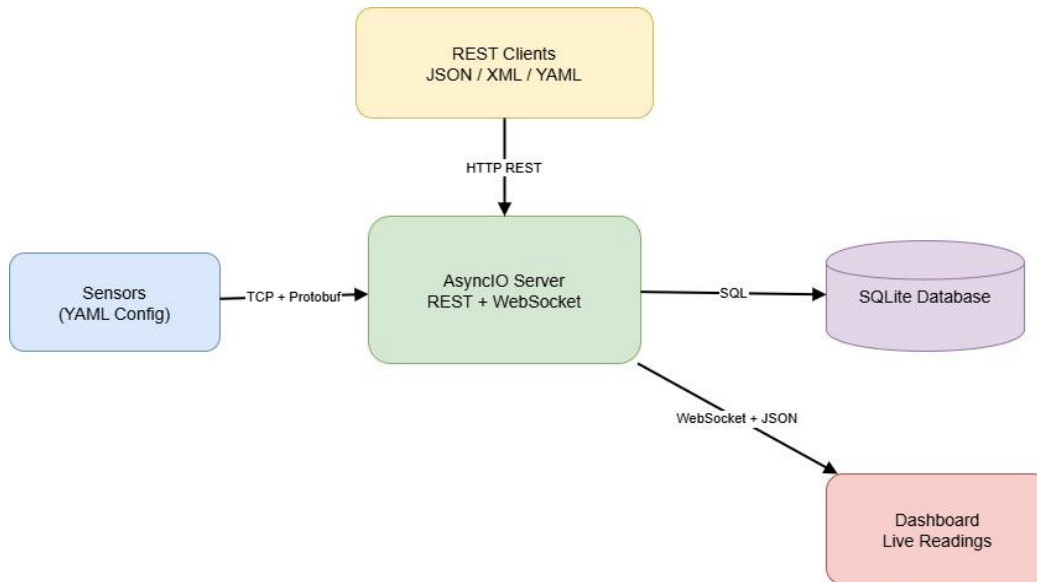


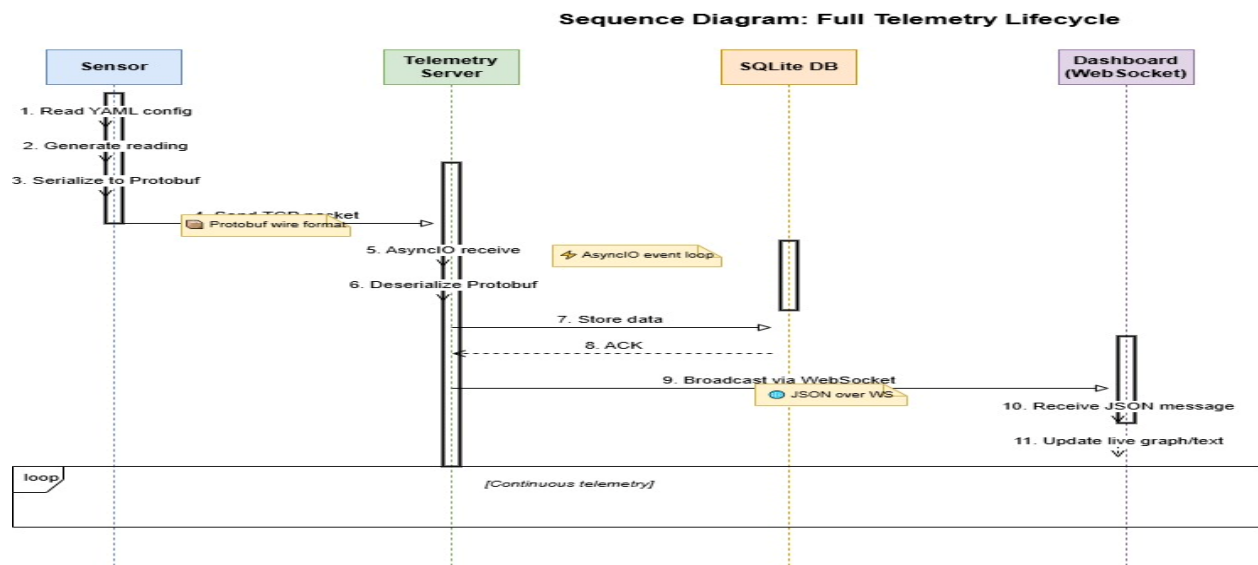
A.SYSTEM ARCHITECTURE DIAGRAM



The AsyncIO server handles both REST and WebSocket connections asynchronously, storing incoming data into an SQLite database.

It then pushes live sensor readings to the dashboard using **WebSocket** with **JSON** formatting. Finally, the dashboard displays these live readings in real time.

B.SEQUENCE DIAGRAM



C.DATA MODEL SCHEMA

```
syntax = "proto3";
package telemetry;

import "google/protobuf/timestamp.proto";

// =====
// Sensor (Persistent Entity)
// =====
message Sensor {
    string sensor_id = 1;
    string location = 2;
    string type = 3;
    string manufacturer = 4;
    bool active = 5;
    google.protobuf.Timestamp registered_at = 6;
}

// =====
// Sensor Reading
// =====
message Reading {
    string sensor_id = 1;
    string measurement_type = 2;
    double value = 3;
    string unit = 4;
    google.protobuf.Timestamp timestamp = 5;
    map<string, string> metadata = 6;
}
```

- This **.proto** file defines the data model for a telemetry system. It contains two main entities: Sensor and Reading.
 - The Sensor message stores information about each physical sensor device such as its ID, location, type, manufacturer, and status.
 - The Reading message stores real-time data collected from sensors, including the measurement value, unit, timestamp, and optional metadata.
- This structure is used to standardize how sensor data is stored and exchanged between systems.

D. .REST ENDPOINT SPECIFICATION

PATH	METHOD	REQUESTED BODY	RESPONSE BODY	STATUS CODE	EXAMPLE
/Sensors	GET	None	List of Sensors	200 OK	[{ "sensor_id" : "s001", "location": "GH1", "type": "temperature" }]
/Sensors	POST	Sensor Object	Created Sensor	201 Created, 400 Bad request	[{ "sensor_id" : "s002", "location": "GH2"}]
/Sensors/{id}readings	GET	None	List of readings	200 OK Not Found	[{ "sensor_id" : "s001", "location": "GH2"}]
/Sensors/{id}	DELETE	None	Success message	200K, 404 Not Found	{ "message" : "sensor deleted successfully" }

E. YAML CONFIGURATION SCHEMA STRUCTURE

```
sensors:
- id: <string>
  type: <string>
  location: <string>
  units: <string>
  polling_interval: <integer>
  thresholds:
    min: <number>
    max: <number>
```

Example: Snippet

```
sensors:
- id: temp1
  type: temperature
  location: "Server Room"
  units: Celsius
  polling_interval: 60
  thresholds:
    min: 15
    max: 30
- id: humidity1
  type: humidity
  location: "Warehouse"
  units: "%RH"
  polling_interval: 120
  thresholds:
    min: 30
    max: 70
```

F. CONCURRENCY MODEL

1. How AsyncIO tasks are organized

- The event loop starts running.
- Coroutines are scheduled as tasks.
- Tasks execute until they reach an `await`.
- While one task waits, the event loop runs another ready task.
- This enables efficient concurrent execution without multiple threads.

2. Key Coroutines and their Roles

- **Producer:** A coroutine that generates items and uses `await queue.put()` to place them into an `asyncio.queue`.
- **Consumer:** A coroutine that uses `await queue.get()` to take items from the queue and process them.
- **Event Manager:** Often the `main()` coroutine, which schedules producers and consumers using `create_task()` and manages their lifecycle.

3. How they communicate

- **`asyncio.Queue`:** The standard mechanism for communication between producers and consumers, providing `put()` and `get()` methods that yield to the event loop.
- **`await / asyncio.gather()`:** used to wait for other tasks or gather results.
- **`asyncio.Event / Future`:** used for signaling or sharing results between coroutines.

4. **When a Slow Consumer Can't Keep Up**, the `asyncio.Queue` will continue to grow, leading to high memory usage hence:
- **Memory Exhaustion:** If the queue is unbounded, it will consume memory until the application crashes.
 - **Delayed Processing:** Items wait longer in the queue, increasing latency for end-to-end processing.

G. FAILURE HANDLING

- Failure handling in modern infrastructure is designed to ensure system resilience through automation, redundancy, and graceful degradation.

Here is what happens in each scenario, along with typical recovery mechanisms:

1. Sensor Disconnects (IoT/Edge Failure)

- Local Data Caching
- Automatic Reconnection
- Fault Detection

2. Storage Layer Fails

- Automatic Failover
- Replication & Redundancy
- Data Consistency

3. Server Overloaded

- Load Shedding
- Circuit Breakers
- Rate Limiting